

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Enzo Marambio Vasquez

10 de septiembre de 2026

20:25

1. Introducción

El presente informe aborda el estudio empírico de dos problemas clásicos en computación: el ordenamiento de arreglos unidimensionales y la multiplicación de matrices cuadradas. Se contrastan algoritmos de ordenamiento (`std::sort`, MergeSort, QuickSort y PatienceSort) junto a algoritmos matriciales (Naive iterativo y Strassen recursivo) sobre diferentes tamaños de entrada, distribuciones y dominios de valores. El propósito central no radica en demostrar analíticamente sus cotas asintóticas conocidas, sino en examinar cómo factores reales de la arquitectura moderna de computadores —tales como la jerarquía de memoria caché, la sobrecarga por asignación dinámica en el *heap* y las optimizaciones a nivel de compilador— modulan, amplifican o contradicen las predicciones teóricas en la práctica.

2. Entorno experimental

Las pruebas experimentales se realizaron en un equipo local bajo el sistema operativo **Microsoft Windows 11 (64-bit)**. En cuanto al hardware, el equipo cuenta con un procesador **AMD Ryzen 7 7435HS**, **24 GB de memoria RAM DDR4** y una unidad de almacenamiento de **512 GB SSD**. Todo el código fuente en C++ se compiló mediante **MinGW-w64 GCC (g++)** bajo el estándar C++17, utilizando la bandera de optimización máxima `-O3` y soporte de hilos `-pthread`. Los scripts auxiliares para generar datos y graficar se ejecutaron en **Python 3.14** apoyados en las bibliotecas NumPy, Pandas y Matplotlib.

Datasets evaluados:

- **Ordenamiento:** Arreglos de tamaño $N \in \{10, 10^3, 10^5, 10^7\}$ con tres disposiciones iniciales (*ascendente*, *descendente* y *aleatorio*) y dos dominios: $D_1 = [0, 9]$ y $D_7 = [0, 10^7]$.
- **Multiplicación de matrices:** Matrices cuadradas con dimensión $N \in \{16, 64, 256, 1024\}$, en variantes *dispersa*, *diagonal* y *densa*, bajo dominios $D_0 = \{0, 1\}$ y $D_{10} = \{0, \dots, 9\}$.
- **Replicabilidad estadística:** Para cada caso se ejecutaron tres muestras independientes (a, b, c), considerando el promedio aritmético de sus tiempos para evitar distorsiones por procesos del sistema.

3. Resultados y Análisis

3.1. Ordenamiento de Arreglos

La [Tabla 1](#) resume los tiempos promedio observados para los tamaños extremos ($N = 10^5$ y $N = 10^7$) en arreglos aleatorios, mientras que la [Fig. 1](#) ilustra las curvas de rendimiento en escala logarítmica.

Cuadro 1: Tiempo de ejecución promedio (ms) para arreglos aleatorios ($N = 10^5$ y $N = 10^7$).

Algoritmo	$N = 10^5 (D_1)$	$N = 10^5 (D_7)$	$N = 10^7 (D_1)$	$N = 10^7 (D_7)$
std::sort	2.11 ms	5.19 ms	173.85 ms	581.70 ms
MergeSort	3.58 ms	6.80 ms	382.94 ms	837.80 ms
QuickSort	2.57 ms	6.54 ms	212.49 ms	701.24 ms
PatienceSort	2.82 ms	7.42 ms	224.80 ms	962.82 ms

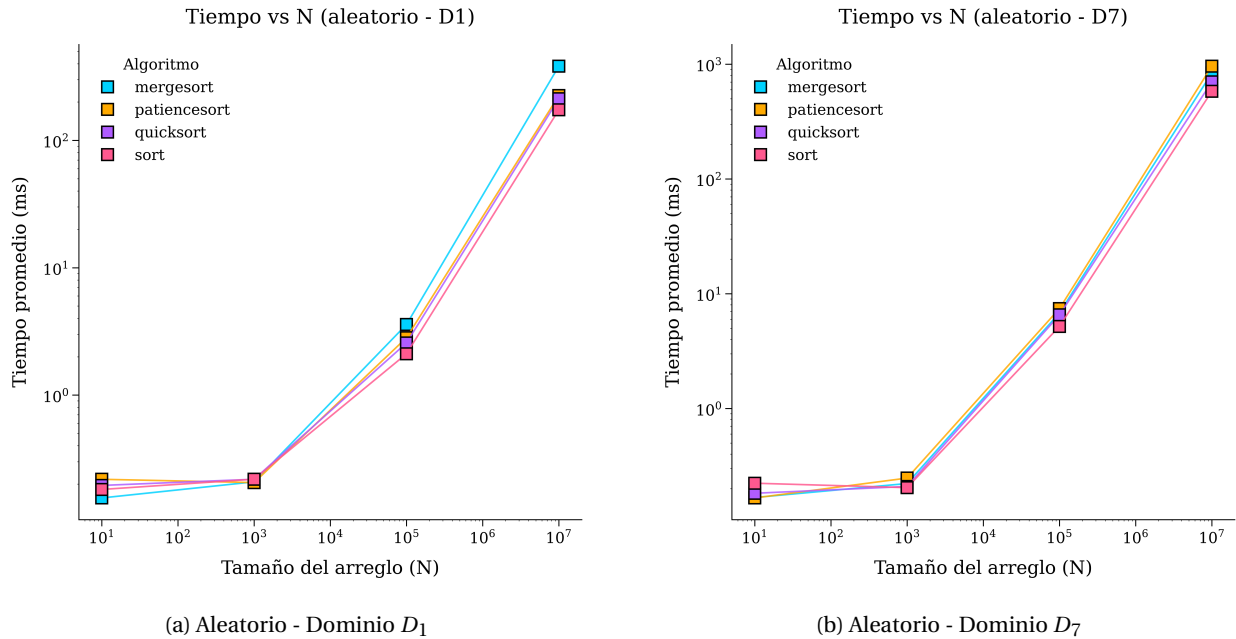


Figura 1: Tiempos de ejecución en ordenamiento para datos aleatorios en dominios D_1 y D_7 .

Como se aprecia empíricamente, `std::sort` (implementación introspectiva híbrida) lidera consis-

tentemente en tiempo de CPU. En el dominio D_1 , donde el rango de valores posibles es diminuto ($[0, 9]$) y abundan las colisiones, todos los algoritmos experimentan una notable reducción de tiempo respecto a D_7 . QuickSort mantiene un desempeño sumamente competitivo gracias a su excelente localidad espacial de caché. Por su parte, PatienceSort exhibe un comportamiento particular: ante entradas ordenadas o con claves altamente concentradas (D_1), el número de pilas que mantiene es bajo, minimizando las comparaciones en la búsqueda binaria; sin embargo, en D_7 con valores crecientes o aleatorios, la cantidad de pilas se aproxima a N , encareciendo la fase de reconstrucción por montículo.

3.2. Multiplicación de Matrices Cuadradas

La [Tabla 2](#) resume los tiempos promedio de ejecución para matrices densas en dominio D_{10} , mientras que la [Fig. 2](#) presenta las curvas comparativas entre ambos algoritmos en escala logarítmica.

Cuadro 2: Tiempo de ejecución promedio (ms) para multiplicación de matrices densas (D_{10}).

Dimensión (N)	Naive (ms)	Strassen (ms)	Ratio (Strassen / Naive)
16	0.0035 ms	1.3288 ms	$\approx 379\times$
64	0.0570 ms	62.3046 ms	$\approx 1093\times$
256	3.5297 ms	3838.94 ms	$\approx 1087\times$
1024	155.53 ms	152541.3 ms	$\approx 980\times$

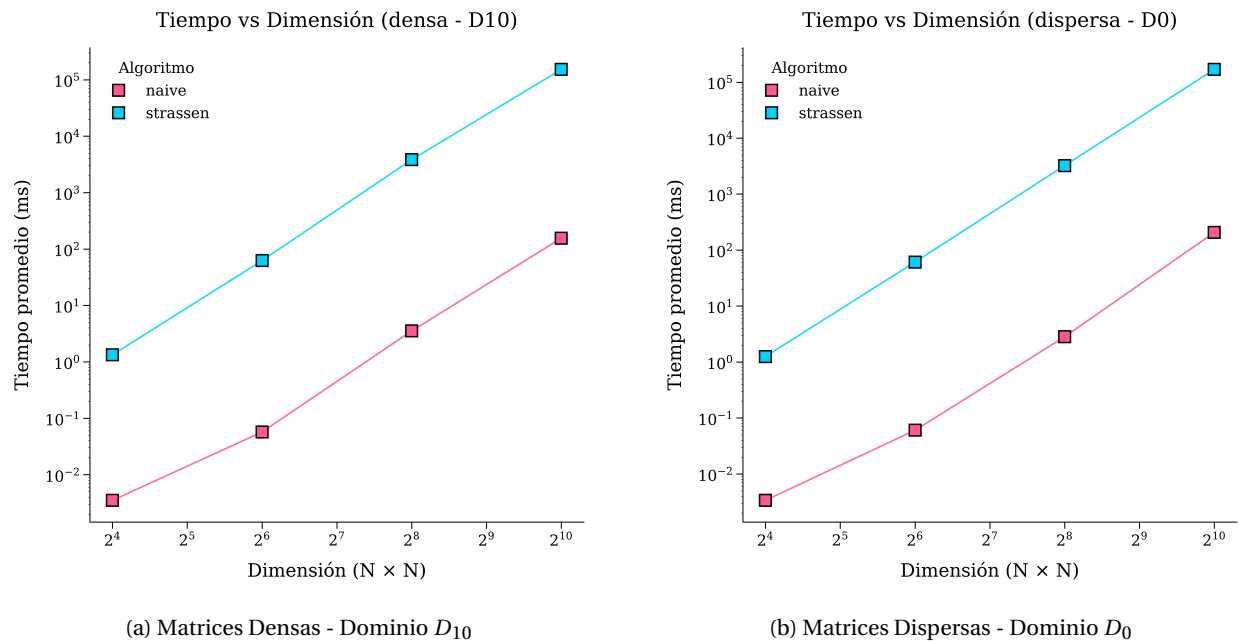


Figura 2: Desempeño empírico en multiplicación matricial (Naive vs. Strassen).

Aunque en la teoría matemática Strassen promete ser más rápido que Naive porque realiza menos multiplicaciones ($\mathcal{O}(N^{2,81})$ contra $\mathcal{O}(N^3)$), en las pruebas prácticas ocurrió lo contrario: Naive fue drásti-

camente más veloz, llegando a ser casi 1000 veces más rápido en matrices de 1024×1024 (0.15 segundos frente a más de 2.5 minutos de Strassen).

Esta gran diferencia se debe principalmente a cómo trabaja la computadora en la realidad:

1. **Naive es simple y ordenado para el procesador:** Utiliza bucles directos sobre memoria continua. Al compilar con la bandera `-O3`, la computadora puede leer y multiplicar varios números al mismo tiempo aprovechando la memoria rápida (caché) sin interrupciones.
2. **Strassen tiene demasiado trabajo extra (sobrecarga):** Al ser recursivo, parte las matrices en pedazos pequeños una y otra vez. En cada división debe crear y borrar decenas de matrices temporales en la memoria RAM. Ese constante proceso de pedir y liberar memoria hace que el procesador gaste la mayor parte del tiempo organizando datos en vez de multiplicarlos.

4. Conclusiones

Los resultados experimentales corroboran que la notación asintótica $\mathcal{O}$ describe la tendencia de crecimiento hacia el infinito, pero no anticipa el desempeño efectivo sobre arquitecturas modernas en instancias acotadas. En ordenamiento, `std::sort` y QuickSort dominan debido a su mínima sobrecarga auxiliar y óptima localidad espacial. En multiplicación de matrices, la formulación directa de Strassen demostró ser inviable frente a Naive (tardando 152 segundos versus 0.15 segundos para $N = 1024$), evidenciando que las constantes ocultas, el tráfico hacia la memoria principal y la falta de un corte híbrido hacia métodos iterativos pueden degradar severamente algoritmos de menor complejidad asintótica.